TECNOLÓGICO UNIVERSITARIO PLAYACAR
INNOVACIÓN CON SENTIDO HUMANO

TECNOLÓGICO UNIVERSITARIO PLAYACAR

INGENIERÍA EN SISTEMAS COMPUTACIONALES

ASIGNATURA:
DESARROLLO Y PROGRAMACIÓN ORIENTADO A OBJETOS 27-1

PROYECTO FINAL (CASO DE ESTUDIO #8):
SISTEMA DE GESTIÓN DE ENVÍOS PARA PAQUETERÍA LOCAL "RIVIERA EXPRESS"

INTEGRANTE DEL EQUIPO	MATRÍCULA
ANGEL RICARDO TEH CATZIN	7433
ALEJANDRO RAMIREZ PEREZ	7507
EVER LOPEZ PANTI	7335
CRISTIAN ALEJANDRO HOIL REYES	7243

DATO ACADÉMICO	INFORMACIÓN
DOCENTE	EMMANUEL ANDRES MARTINEZ LOPEZ
CARRERA	Ing. Sistemas Computacionales
CUATRIMESTRE	7MO Cuatrimestre
FECHA DE ENTREGA	4 de noviembre de 2026

1. PLANTEAMIENTO DEL PROBLEMA

La empresa de envíos urbanos **"Riviera Express"** transporta paquetes y documentos pequeños dentro de la ciudad, pero carece de una herramienta de software que estandarice el cálculo de tarifas y controle sus entregas diarias. El registro manual propicia errores como la captura de pesos, volúmenes o distancias negativas y dificulta organizar varios paquetes dentro de una misma ruta con un repartidor asignado.

Alcance de la solución: Desarrollar un sistema bajo el paradigma de Programación Orientada a Objetos en Python que calcule el costo de envío según peso, volumen y distancia, valide mediante encapsulamiento que no existan medidas negativas, permita operar costos con sobrecarga de los cuatro operadores aritméticos (+, -, *, /) y agrupe los envíos en una hoja de ruta con su repartidor responsable.

2. FASE 1: ESPECIFICACIÓN DE REQUISITOS

2.1. Requisitos Funcionales (RF)

ID	Requisito Funcional	Descripción	Justificación Técnica
RF-01	Registro de envíos	Registrar paquetes (con volumen) y documentos (con urgencia) con su código, peso y distancia.	Aplica herencia desde la superclase Envio hacia las subclases Paquete y Documento.
RF-02	Validación de medidas positivas	Validar que peso, volumen y distancia sean mayores a cero, impidiendo negativos.	Garantiza integridad de datos usando encapsulamiento (get/set y @property).
RF-03	Cálculo de costo de envío	Calcular la tarifa según peso, distancia y volumen o recargo de urgencia.	Automatiza el cobro aplicando polimorfismo en calcular_costo().
RF-04	Operaciones con sobrecarga	Operar costos entre envíos mediante los operadores +, -, * y /.	Cumple la sobrecarga de los 4 operadores (__add__, __sub__, __mul__, __truediv__).
RF-05	Agrupación en Hoja de Ruta	Agrupar varios paquetes y documentos bajo una misma hoja de ruta con folio y zona.	Centraliza los objetos de envío en una lista privada dentro de HojaRuta.

ID	Requisito Funcional	Descripción	Justificación Técnica
RF-06	Asignación de repartidor	Registrar repartidores y asignar uno como responsable de la hoja de ruta.	Vincula un objeto Repartidor a la HojaRuta mediante una propiedad @property.

2.2. Requisitos No Funcionales (RNF)

ID	Requisito No Funcional	Especificación	Justificación Técnica
RNF-01	Modularidad (1 Clase = 1 Archivo)	Cada una de las 5 clases reside en su propio archivo .py y se instancia en main.py.	Cumple la métrica Clases (5) >= Integrantes (4) y separa responsabilidades.
RNF-02	Encapsulamiento mixto	Uso de 2 decoradores @property y métodos get/set tradicionales en atributos privados.	Protege los atributos sensibles contra accesos o cambios inválidos.
RNF-03	Sin manejo de excepciones	Validación con condicionales if/else, sin usar bloques try/except ni lanzar errores.	Asegura una ejecución limpia de principio a fin conforme a las reglas del curso.
RNF-04	Código sin comentarios	Archivos .py limpios, autodocumentados y sin líneas comentadas (#).	Cumple la restricción de entrega de código limpio sin comentarios intermedios.

3. FASE 2: ANÁLISIS ORIENTADO A OBJETOS

3.1. Lista de Objetos Identificados

Clase / Objeto	Atributos (+ Público / - Privado)	Responsabilidades y Métodos Principales
Envio (Superclase)	+ codigo - __peso - __distancia	Constructor y destructor, validación de peso y distancia (get/set), cálculo de costo base y sobrecarga de +, -, *, /.
Paquete (Subclase)	- __volumen (Hereda de Envio)	Hereda de Envio, encapsula volumen con @property evitando negativos y suma el volumen al costo total.
Documento (Subclase)	+ urgente (Hereda de Envio)	Hereda de Envio y suma un cargo adicional al costo base cuando el documento es urgente.
Repartidor	+ nombre + vehiculo - __id_repartidor	Representa al repartidor asignado a las entregas y protege su identificador con get_id_repartidor() y set_id_repartidor().
HojaRuta	+ folio, + zona - __repartidor - __envios	Agrupa varios paquetes y documentos, asigna al repartidor mediante @property y muestra el resumen de la ruta.

3.2. Diagrama de Casos de Uso

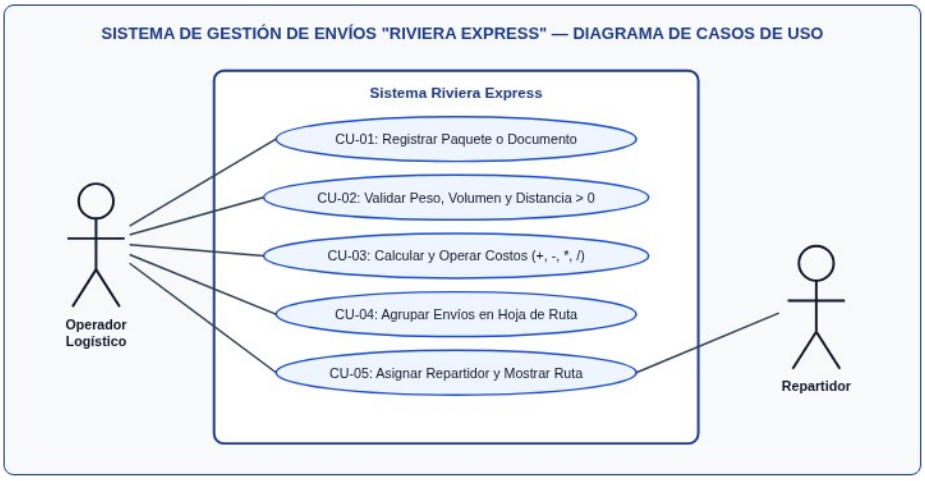


Figura 1. Diagrama de Casos de Uso del Sistema Riviera Express.

4. FASE 3: DISEÑO ORIENTADO A OBJETOS

4.1. Diagrama de Clases UML (+ Público / - Privado)

El siguiente diagrama de clases UML representa las 5 clases del sistema (superando el mínimo de 4 integrantes del equipo), mostrando la visibilidad pública (**+**) y privada (**-**), la relación de herencia hacia **Envio** y la agrupación en **HojaRuta**:

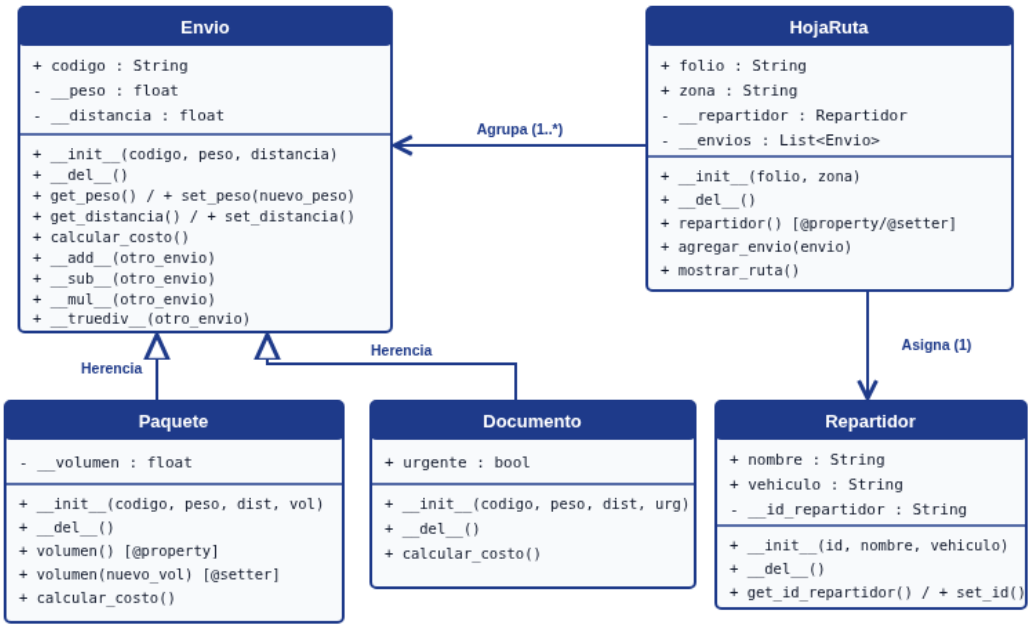


Figura 2. Diagrama de Clases UML del Sistema Riviera Express.

5. FASE 4: IMPLEMENTACIÓN Y EVIDENCIAS

5.1. Evidencia de las Clases Implementadas (1 Archivo = 1 Clase)

Envio.py

```
class Envio:
    def __init__(self, codigo, peso, distancia):
        self.codigo = codigo
        self.__peso = peso
        self.__distancia = distancia

    def __del__(self):
        print(f"Envio {self.codigo} eliminado")

    def get_peso(self):
        return self.__peso

    def set_peso(self, nuevo_peso):
        if nuevo_peso > 0:
            self.__peso = nuevo_peso
        else:
            print("El peso no puede ser negativo")

    def get_distancia(self):
        return self.__distancia

    def set_distancia(self, nueva_distancia):
        if nueva_distancia > 0:
            self.__distancia = nueva_distancia
        else:
            print("La distancia no puede ser negativa")

    def calcular_costo(self):
        return (self.__peso * 10) + (self.__distancia * 5)

    def __add__(self, otro_envio):
        return self.calcular_costo() + otro_envio.calcular_costo()

    def __sub__(self, otro_envio):
        return self.calcular_costo() - otro_envio.calcular_costo()

    def __mul__(self, otro_envio):
        return self.calcular_costo() * otro_envio.calcular_costo()

    def __truediv__(self, otro_envio):
        return self.calcular_costo() / otro_envio.calcular_costo()
```

Paquete.py

```
from Envio import Envio


class Paquete(Envio):
    def __init__(self, codigo, peso, distancia, volumen):
        super().__init__(codigo, peso, distancia)
        self.__volumen = volumen

    def __del__(self):
        print(f"Paquete {self.codigo} eliminado")

    @property
    def volumen(self):
        return self.__volumen

    @volumen.setter
    def volumen(self, nuevo_volumen):
        if nuevo_volumen > 0:
            self.__volumen = nuevo_volumen
        else:
            print("El volumen no puede ser negativo")

    def calcular_costo(self):
        return super().calcular_costo() + (self.__volumen * 20)
```

Documento.py

```
from Envio import Envio


class Documento(Envio):
    def __init__(self, codigo, peso, distancia, urgente):
        super().__init__(codigo, peso, distancia)
        self.urgente = urgente

    def __del__(self):
        print(f"Documento {self.codigo} eliminado")

    def calcular_costo(self):
        costo = super().calcular_costo()
        if self.urgente:
            return costo + 30
        return costo
```

Repartidor.py

```
class Repartidor:
    def __init__(self, id_repartidor, nombre, vehiculo):
        self.nombre = nombre
        self.vehiculo = vehiculo
        self.__id_repartidor = id_repartidor

    def __del__(self):
        print(f"Repartidor {self.nombre} eliminado")

    def get_id_repartidor(self):
        return self.__id_repartidor
```

HojaRuta.py

```
class HojaRuta:
    def __init__(self, folio, zona):
        self.folio = folio
        self.zona = zona
        self.__repartidor = None
        self.__envios = []

    def __del__(self):
```

Figura 3. Código Fuente.

5.2. Evidencia de Ejecución y Validación de Resultados (main.py)



*Figura 4. Salida en consola validando herencia, encapsulamiento, sobrecarga (+, -, *, /), hoja de ruta y destructores.*